AEGIS SOC DASHBOARD

AI-Enhanced Security Operations Center for Real-Time Network Threat Detection and Automated Defense

---

PROJECT BOOK

Submitted in Partial Fulfillment of the Requirements for

Internship / Final Year Project

Department of Computer Science & Engineering

Academic Year 2025 – 2026

ABSTRACT

This project presents AEGIS (Automated Enforcement and Guardian Intelligence System), a real-time Security Operations Center (SOC) dashboard designed and implemented for a simulated corporate network environment. AEGIS integrates multi-source security event ingestion, AI-powered threat analysis, automated firewall defense, and live network visualization into a unified web-based platform.

The system monitors four core company servers — a web server (Apache2), a DNS server (BIND9), a customer database (MySQL), and an LDAP authentication server (OpenLDAP) — alongside network-level intrusion detection through pfSense/Suricata IDS. Security events from Fail2ban, SSH logs, application logs, and Suricata EVE JSON are forwarded in real time by the AEGIS Forwarder Agent to a centralized Express.js API server backed by Supabase (PostgreSQL).

The dashboard, built with React and Vite, provides a live threat map, event feed, defense rule engine, AI threat briefing via Groq (LLaMA 3.3-70B), automated report generation, and Telegram push notifications. The auto-defense engine evaluates incoming events against configurable rules and dispatches firewall commands to pfSense and Ubuntu VMs without human intervention.

The entire infrastructure is virtualized in GNS3 using a layered network topology with separate DMZ, Internal, and Management VLANs managed by a pfSense firewall. Attack simulations using Kali Linux tools (nmap, hydra, sqlmap, hping3, Metasploit) successfully triggered detection, alerting, and automated response across all monitored services.

Keywords: SOC, IDS, SIEM, Auto-defense, GNS3, pfSense, Suricata, Fail2ban, React, Supabase, Groq AI, Network Security, Intrusion Detection

Abstract (Myanmar)

ဤ project သည် Golden Myanmar Trading Co., Ltd. ၏ ကွန်ပျူတာကွန်ရက်[illegible]ကုမ္ပဏီ ကို network ပုံစံဖြင့် GNS3 virtualization ပတ်ဝန်းကျင်[illegible]ဆောက်က Security Operations Center (SOC) အနေဖြင့် real-time monitoring, threat detection, automated defense တို့ကို ပေင်စပ်ထ သည့် AEGIS dashboard ကို တင်ဆက်သည်။ ကျောင်သမျ၏ internship final project အဖြစ် cybersecurity အခြေခံမျ၊ network topology design, web development, AI integration တို့ကို လက်တွေ့ကျကျ အကောင်အထည်ဖေထသေကြောင့် academic တန်ဖိုနှင့် industry relevance နှစ်ရပ်လုံ ပြည့်ဝသည်။

ACKNOWLEDGEMENTS

First and foremost, I would like to express my deepest and most sincere gratitude to my project supervisor, Dr. Htay Htay Yi, for her exceptional guidance, continuous encouragement, and invaluable technical insights throughout every phase of this project. Her constructive feedback and unwavering support were instrumental in shaping the direction and quality of this work.

I am also profoundly grateful to Dr. Thiri Thitsar Khaing for her dedicated teaching, academic mentorship, and patient guidance in helping me understand the theoretical foundations of network security and system design that underpin this project.

My sincere appreciation also goes to Dr. Thu Zar San for her thoughtful advice, encouragement throughout the research process, and for sharing her expertise in cybersecurity concepts and best practices that greatly enriched the depth and rigor of this work.

I am deeply thankful to my internship organization for providing the necessary resources, virtual machine infrastructure, and technical mentorship that made the implementation of a real-world SOC environment possible.

Special thanks go to the open-source communities behind Suricata, Fail2ban, pfSense, React, Supabase, and GNS3, whose free tools formed the backbone of this system.

I also acknowledge the Groq AI platform for providing free API access to the LLaMA 3.3-70B model, which powers the AI threat analysis features of AEGIS.

Finally, I would like to thank my family and friends for their moral support and encouragement throughout this challenging yet rewarding project.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AEGIS	Automated Enforcement and Guardian Intelligence System
AI	Artificial Intelligence
API	Application Programming Interface
BIND9	Berkeley Internet Name Domain version 9
CLI	Command Line Interface
CSRF	Cross-Site Request Forgery
DDoS	Distributed Denial of Service
DMZ	Demilitarized Zone
DNS	Domain Name System
EVE	Extensible Event Format (Suricata JSON output)
FTP	File Transfer Protocol
GNS3	Graphical Network Simulator 3
HTTP	HyperText Transfer Protocol
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
JSON	JavaScript Object Notation
JWT	JSON Web Token
LDAP	Lightweight Directory Access Protocol
LFI	Local File Inclusion
LLM	Large Language Model
MGMT	Management
MITM	Man-In-The-Middle (attack)
MySQL	My Structured Query Language (database)
NAT	Network Address Translation
PHP	PHP: Hypertext Preprocessor
REST	Representational State Transfer
RFI	Remote File Inclusion
SIEM	Security Information and Event Management
SOC	Security Operations Center
SQL	Structured Query Language
SQLi	SQL Injection
SSH	Secure Shell
SSL	Secure Sockets Layer
SSO	Single Sign-On
TLS	Transport Layer Security
UI	User Interface
URL	Uniform Resource Locator
VLAN	Virtual Local Area Network
VM	Virtual Machine
VoIP	Voice over Internet Protocol
WAF	Web Application Firewall
XSS	Cross-Site Scripting

CHAPTER 1: INTRODUCTION

1.1 Background

In the modern digital landscape, organizations of every size face an ever-growing volume of cyber threats. From small businesses to large enterprises, network infrastructure is continuously targeted by attackers exploiting vulnerabilities in web applications, databases, authentication systems, and network protocols. The cost of a successful breach — in data loss, financial damage, and reputational harm — can be catastrophic.

A Security Operations Center (SOC) serves as the nerve center of an organization's cybersecurity posture. It is a centralized unit responsible for monitoring, detecting, analyzing, and responding to security incidents in real time. Traditionally, SOC operations require significant investment in commercial SIEM platforms such as Splunk, IBM QRadar, or Microsoft Sentinel. For mid-sized organizations and academic purposes, building an open-source SOC that achieves comparable functionality at minimal cost represents a compelling challenge.

This project, AEGIS (Automated Enforcement and Guardian Intelligence System), addresses that challenge. It was conceived and built as a final internship project to demonstrate that a production-quality SOC can be constructed entirely from open-source components: pfSense for firewall and routing, Suricata for network-level IDS, Fail2ban for host-based intrusion prevention, and a custom React + Express.js dashboard for centralized visualization and control.

AEGIS goes beyond simple log aggregation. It implements an automated defense pipeline that translates detected threat events into executable firewall commands — without human intervention. It integrates AI (Groq LLaMA 3.3-70B) for intelligent threat briefing and natural language explanations of security events. And it delivers real-time push notifications via Telegram, ensuring the security team is immediately informed of critical incidents regardless of whether the dashboard is open.

1.2 Problem Statement

Small-to-medium enterprises and academic institutions often lack the budget and personnel to operate a fully-featured SOC. Commercial SIEM solutions are expensive, complex to deploy, and require continuous tuning. Free alternatives are often fragmented — requiring administrators to manually correlate logs from disparate sources such as Fail2ban, Suricata, Apache, and SSH.

The specific problems this project addresses are:

1. Log data from multiple security tools (Fail2ban, Suricata, Apache access logs, SSH auth logs, MySQL logs, BIND9 logs, OpenLDAP logs) is scattered across multiple servers with no unified visibility.
2. Security administrators must manually review logs to detect attacks — a slow, error-prone process that allows threats to persist undetected.
3. When an attack is detected, manual firewall rule creation is slow and may lag behind fast-moving threats such as port scans or brute-force attacks.
4. There is no centralized way to visualize the network topology, see which nodes are under attack, or understand the severity and nature of threats at a glance.
5. Generating periodic security reports and threat briefings requires manual effort and security expertise that may not be available in all organizations.

AEGIS solves all five problems through a unified, real-time, AI-augmented SOC dashboard with automated defense capabilities.

1.3 Objectives

The objectives of this project are:

- To design and implement a multi-source security event collection system that ingests events from Fail2ban, Suricata IDS, SSH logs, Apache access logs, MySQL logs, BIND9 logs, and OpenLDAP logs in real time.
- To build a centralized Express.js API server with secure ingestion endpoints and a Supabase (PostgreSQL) backend for persistent event storage.
- To develop a React-based SOC dashboard with live threat visualization, event feed, network topology map, and security KPI cards.
- To implement an auto-defense engine that automatically evaluates incoming events against configurable defense rules and dispatches firewall commands to pfSense and Ubuntu VMs.
- To integrate Groq AI (LLaMA 3.3-70B) for intelligent threat analysis, event explanation, and automated report generation.
- To provide real-time Telegram push notifications for critical and high-severity security events.
- To validate the complete system through realistic attack simulations using Kali Linux in a GNS3 virtual lab environment.

1.4 Scope of the Project

The scope of this project covers the design, implementation, and testing of the AEGIS SOC Dashboard within a GNS3-virtualized network environment representing Golden Myanmar Trading Co., Ltd.'s internal infrastructure.

In scope:

- Four company servers: web server, DNS server, customer database, LDAP server
- pfSense firewall with Suricata IDS covering all network zones
- AEGIS Forwarder Agent running on a dedicated Ubuntu hub VM
- Full-stack SOC dashboard (React frontend + Express.js backend)

- Auto-defense engine with configurable rules
- AI-powered threat analysis and report generation
- Telegram notification integration
- Kali Linux-based attack simulation and validation

Out of scope:

- Physical network deployment (all infrastructure is virtualized in GNS3)
- Email server, VoIP server, CCTV simulation (planned for future phases)
- Active Directory / Samba4 integration (future phase)
- Mobile application (dashboard is web-only)

1.5 Report Organization

The remainder of this report is organized as follows:

- Chapter 2 reviews existing literature on SOC, IDS, SIEM, and AI in cybersecurity.
- Chapter 3 presents system analysis, requirements, and design decisions.
- Chapter 4 describes the full implementation of all system components.
- Chapter 5 documents testing, attack simulations, and results.
- Chapter 6 discusses current limitations and future enhancements.
- Chapter 7 concludes the report.
- Appendices provide installation, API, and configuration references.

1.6 Research Methodology

The development of AEGIS followed a structured research and development methodology to ensure technical rigor and system reliability:

- Phase 1: Literature Review — Analyzing existing SOC/SIEM architectures and open-source security tools.
- Phase 2: Requirements Analysis — Defining functional and non-functional needs for a mid-sized corporate lab.
- Phase 3: Network Design — Designing a multi-zone GNS3 topology with DMZ, Internal, and Management VLANs.
- Phase 4: Implementation — Developing the full-stack dashboard, API server, and Python forwarder agent.
- Phase 5: Integration — Connecting the GNS3 virtual machines to the cloud-hosted AEGIS platform.
- Phase 6: Testing and Evaluation — Simulating real-world attacks using Kali Linux and validating the auto-defense response.

CHAPTER 2: LITERATURE REVIEW

2.1 Security Operations Center (SOC)

A Security Operations Center (SOC) is a centralized facility where a security team monitors, detects, contains, and responds to cybersecurity threats and incidents. The primary function of a SOC is continuous monitoring of an organization's IT infrastructure — networks, servers, endpoints, databases, applications, websites, and other technology — for signs of a security incident.

According to NIST Special Publication 800-61, an effective security operations capability must include: event monitoring and analysis, incident detection and classification, incident response coordination, threat intelligence integration, and continuous improvement through lessons learned. Modern SOCs increasingly incorporate automation (SOAR — Security Orchestration, Automation and Response) to handle the volume of alerts that human analysts alone cannot process.

AEGIS implements the core SOC functions at a scale appropriate for a mid-sized organization: event collection from multiple sources, severity-based classification, automated response, and human-readable threat briefing. While it does not implement full SOAR capabilities, the auto-defense engine represents a lightweight SOAR component for the most common threat types.

2.2 Intrusion Detection Systems (IDS)

An Intrusion Detection System (IDS) monitors network traffic or host activity for malicious patterns and generates alerts when such patterns are identified. IDS solutions are broadly categorized as:

- Network-based IDS (NIDS): inspects network traffic at a strategic point. Examples include Snort and Suricata.
- Host-based IDS (HIDS): monitors activity on an individual host. Examples include OSSEC, Wazuh, and Fail2ban.
- Hybrid: combines both approaches.

Suricata, developed by the Open Information Security Foundation (OISF), is a high-performance open-source IDS/IPS engine. It supports multi-threading, hardware acceleration, and outputs alerts in EVE JSON format — a structured log format that AEGIS consumes directly.

Fail2ban is a host-based intrusion prevention tool that monitors system log files for repeated authentication failures and automatically updates firewall rules to block offending IP addresses. AEGIS monitors Fail2ban's ban/unban actions across all four company VMs, treating each Fail2ban action as a structured security event.

2.3 Security Information and Event Management (SIEM)

SIEM systems combine Security Information Management (SIM) — long-term storage and analysis of log data — with Security Event Management (SEM) — real-time monitoring and correlation of events. The defining capability of a SIEM is correlation: detecting complex attack patterns that span multiple systems, timeframes, and event types.

Commercial SIEM solutions include Splunk Enterprise Security, IBM QRadar, Microsoft Sentinel, and Elastic SIEM. Open-source alternatives include Wazuh (which builds on OSSEC) and the ELK Stack (Elasticsearch, Logstash, Kibana).

AEGIS does not aim to replace a full SIEM but provides the most critical SIEM capabilities — event collection, normalization, storage, real-time alerting, and basic correlation through its defense rule engine — without the deployment complexity and licensing costs of commercial solutions.

2.4 Network Virtualization with GNS3

GNS3 (Graphical Network Simulator 3) is an open-source network simulation platform that allows the emulation of complex networks using real network operating system images. Unlike purely simulated environments, GNS3 runs actual OS images (Ubuntu, pfSense, MikroTik RouterOS) on the host machine's CPU, making the network behavior indistinguishable from physical hardware.

For this project, GNS3 provides the complete virtual lab: four Ubuntu VMs for company services, one pfSense VM for firewall and IDS, one MikroTik CHR for WAN routing, one Kali Linux VM for attack simulation, and one Ubuntu hub VM for the AEGIS Forwarder Agent. This architecture allows realistic attack-defense scenarios that would be impossible to replicate in a pure software simulation.

2.5 AI in Cybersecurity

The application of Large Language Models (LLMs) to cybersecurity has grown rapidly since 2023. LLMs can translate raw security event data into human-readable explanations, recommend countermeasures, and generate executive-level threat briefings — capabilities that previously required experienced security analysts.

AEGIS integrates Groq's API, which provides inference on Meta's LLaMA 3.3-70B model at extremely low latency. The model is prompted with structured event data from the past 24 hours and generates threat briefings in a mixed Burmese-English format suitable for the local analyst audience. Groq's hardware-accelerated inference (LPU chips) enables sub-second response times even for 70B parameter models.

2.6 Related Works

Several open-source SOC projects have been developed in academic and research contexts. Wazuh (formerly OSSEC) provides a comprehensive HIDS and SIEM solution with a Kibana-based dashboard. Security Onion combines Suricata, Zeek, and Elastic SIEM into a distribution focused on network security monitoring. TheHive provides incident response case management.

AEGIS differentiates itself from these solutions in three key ways: (1) it is purpose-built for a specific network topology and attack scenario set, allowing deeper integration with the monitored services; (2) it includes an auto-defense engine that issues actual firewall commands rather than just alerting; and (3) it integrates generative AI for natural language threat analysis, which none of the above solutions natively provide.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Functional Requirements

The following functional requirements were identified through analysis of the target environment and stakeholder needs:

ID	Requirement	Priority
FR-01	System shall ingest security events from Fail2ban on all four VMs	High
FR-02	System shall ingest SSH authentication events (success and failure)	High
FR-03	System shall ingest Suricata IDS alerts from pfSense EVE JSON	High
FR-04	System shall ingest Apache2 access logs from company-web-server	High
FR-05	System shall ingest MySQL login events from company-customer-db	Medium
FR-06	System shall ingest BIND9 DNS query/error events	Medium
FR-07	System shall ingest OpenLDAP authentication events	Medium
FR-08	Dashboard shall display live event feed with severity indicators	High
FR-09	Dashboard shall show a live network topology/threat map	High
FR-10	System shall auto-evaluate events against defense rules	High
FR-11	System shall dispatch firewall commands to VMs automatically	High
FR-12	Dashboard shall support manual IP block/unblock operations	High
FR-13	System shall send Telegram notifications for critical/high events	High
FR-14	System shall generate AI-powered threat briefings	Medium
FR-15	System shall generate periodic security reports	Medium
FR-16	Dashboard shall authenticate admins via JWT + Google SSO	High

3.2 Non-Functional Requirements

- Security: All ingest endpoints protected by API key authentication. Admin endpoints protected by separate admin key. JWT tokens expire in 24 hours.
- Availability: Dashboard hosted on Vercel CDN; API server on Render. Target uptime ≥ 99.5%.

- Performance: Event-to-dashboard latency ≤ 3 seconds. Auto-defense command execution ≤ 30 seconds from event ingestion.
- Scalability: Event storage in Supabase PostgreSQL; supports millions of rows.
- Maintainability: All forwarder configuration in external local.conf file; no code changes needed to reconfigure VM IPs or log paths.

3.3 Overall System Architecture

AEGIS follows a three-tier architecture separating data collection, processing, and presentation layers:

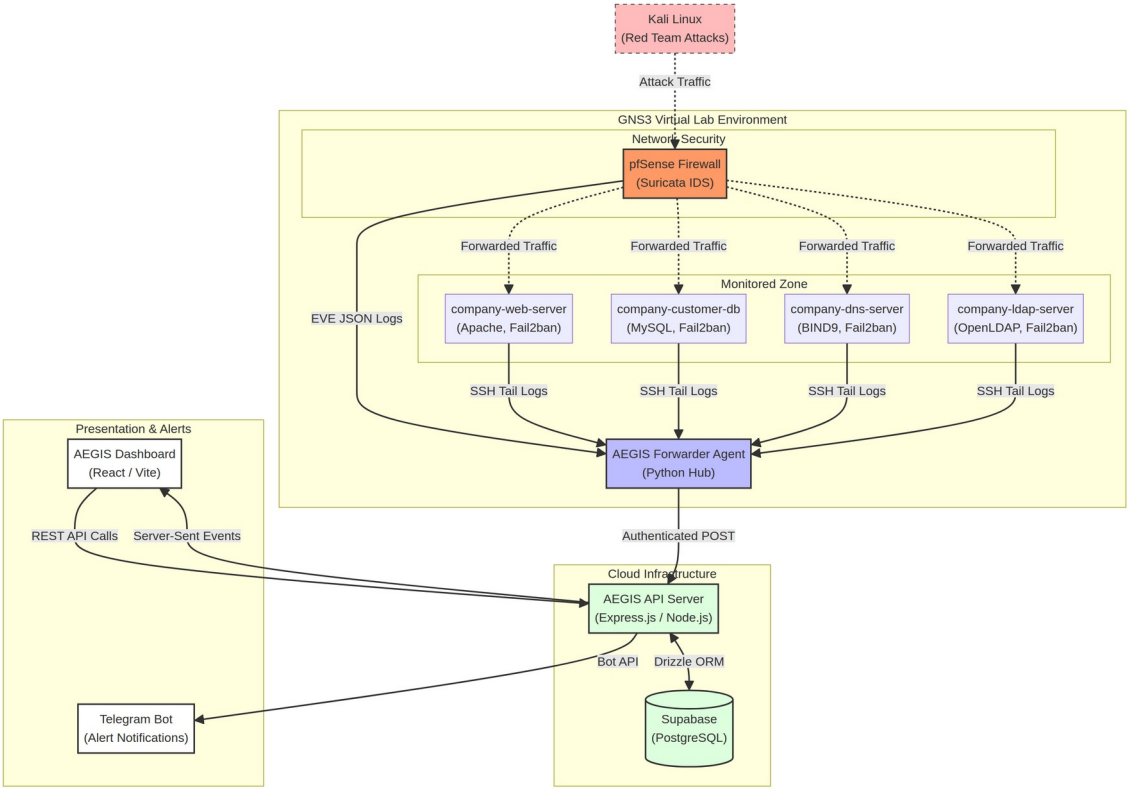

Figure 3.1: Overall AEGIS System Architecture

Tier 1 — Data Collection Layer:

- AEGIS Forwarder Agent (Python) runs on the hub VM (10.30.30.10)
- Spawns independent monitoring threads for each service/VM combination
- Normalizes raw log data into structured JSON events
- Forwards events via authenticated HTTPS POST to the API server

Tier 2 — Processing and Storage Layer:

- Express.js API server (Node.js/TypeScript) hosted on Render
- Receives, validates, and stores events in Supabase PostgreSQL
- Evaluates events against defense rules (auto-defense engine)
- Serves SSE (Server-Sent Events) stream to connected dashboard clients
- Sends Telegram notifications for critical/high events

Tier 3 — Presentation Layer:

- React + Vite dashboard hosted on Vercel
- Consumes SSE stream for real-time updates (no polling)
- Calls REST API for historical data, defense management, and AI analysis

3.4 Network Topology Design

The virtual network is organized into four logical zones separated by pfSense firewall interfaces:

Zone	VLAN/Interface	Subnet	Hosts
DMZ (Public)	pfSense em1	10.10.10.0/24	company-web-server, company-dns-server
Internal	pfSense em2	10.20.20.0/24	company-customer-db, company-ldap-server
Management	pfSense em3	10.30.30.0/24	aegis-company-admin (hub)
WAN Link	pfSense e0 / Router e2	10.0.23.0/30	pfSense WAN, Router
Attacker	Router e1	192.168.10.0/24	Kali Linux (DHCP)
Internet	Router e0 / virbr0	192.168.122.0/24	GNS3 NAT cloud

All inter-zone traffic passes through pfSense, which serves as both the network gateway and the IDS sensor point. Suricata on pfSense inspects traffic on the DMZ and Internal interfaces, providing network-level intrusion detection for all four company servers without requiring agents on each VM.

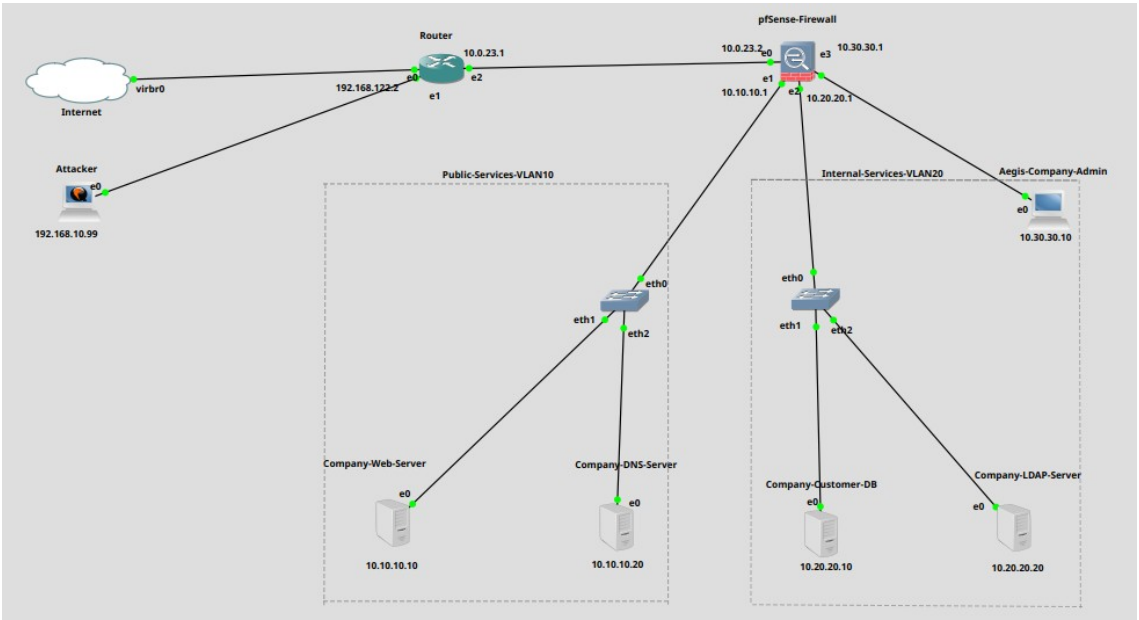

Figure 3.2: GNS3 Network Topology Diagram

3.5 Database Schema Design

The AEGIS database is hosted on Supabase (managed PostgreSQL). The core tables are:

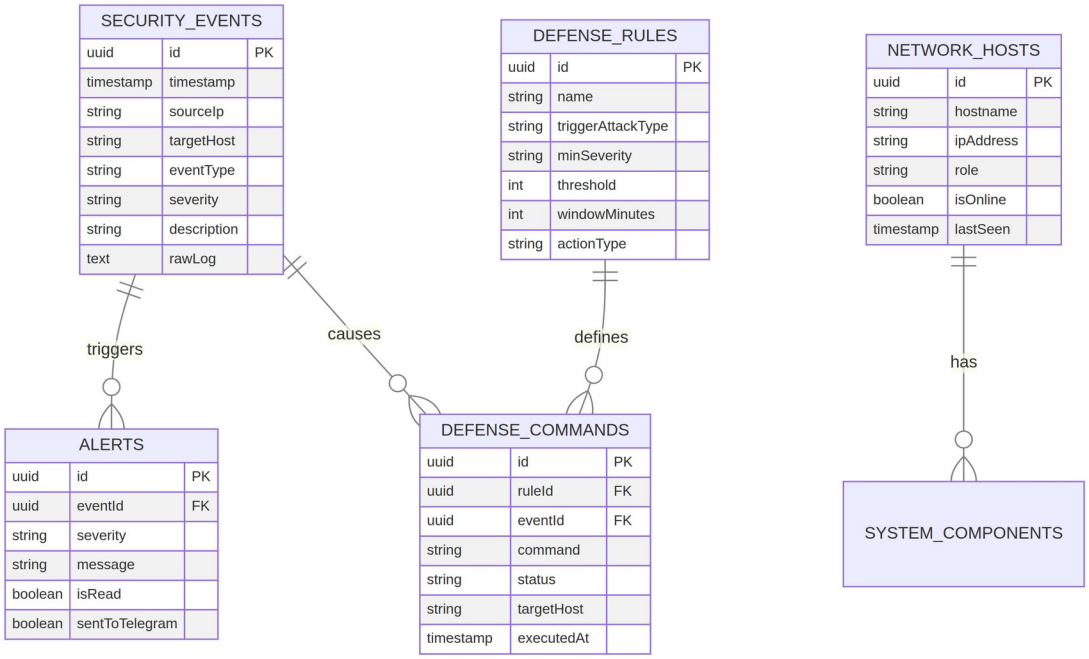

Figure 3.3: Database Entity-Relationship Diagram

Table	Purpose	Key Columns
security_events	All ingested security events	id, timestamp, sourceIp, targetHost, eventType, severity, description, rawLog, signature
alerts	Dashboard alert cards linked to events	id, eventId, severity, message, isRead, sentToTelegram
defense_rules	Auto-defense trigger rules	id, name, triggerAttackType, minSeverity, threshold, windowMinutes, actionType, commandTemplate
defense_commands	Pending/executed firewall commands	id, ruleId, eventId, command, status, targetHost, executedAt
blocked_ips	Currently blocked IP addresses	id, ipAddress, reason, blockedAt, blockedBy, expiresAt
firewall_rules	Manual firewall rules	id, name, action, protocol, sourceIp, destinationIp, port, isActive
network_hosts	Registered network hosts	id, hostname, ipAddress, role, isOnline, lastSeen
system_components	VM service health status	id, hostIp, componentName, status, lastChecked
reports	Generated security reports	id, generatedAt, period, totalEvents, summary, aiGenerated

3.6 API Architecture

The API server exposes three categories of endpoints:

- Ingest endpoints (/api/ingest/*): Accept POST requests from the AEGIS Forwarder Agent. Protected by AEGIS_INGEST_KEY header. One endpoint per event type: /ingest/fail2ban,

/ingest/ssh, /ingest/http, /ingest/mysql, /ingest/dns, /ingest/ldap, /ingest/suricata, /ingest/event.

- Admin endpoints (/api/defense/*, /api/firewall/*, /api/reports/*): Protected by AEGIS_ADMIN_KEY header. Used by the dashboard for defense management and report generation.
- Public endpoints (/api/events, /api/alerts, /api/hosts, /api/ai/*, /api/stream): JWT-protected. Used by the dashboard for data display.

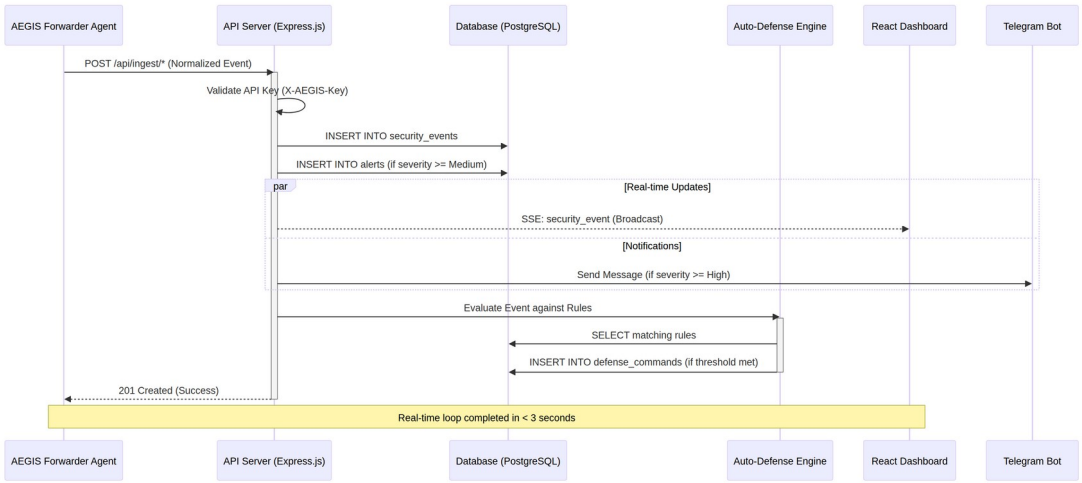

Figure 3.4: API Server Request-Response Flow

3.7 Dashboard UX Design

The dashboard follows a dark-mode-first design with a sidebar navigation and a top Viewing bar showing the current device context. Key design decisions:

- Dark theme: reduces eye strain during 24/7 SOC monitoring. Colors follow severity semantics: red = critical, orange = high, yellow = medium, blue = low, gray = info.
- Real-time updates via SSE: no page refresh needed; new events appear instantly in the event feed and trigger dashboard alerts.
- Threat Map: animated D3.js/SVG network topology showing live attack packets as colored moving dots between nodes.
- Global attack bar: when a new security event arrives, the Viewing bar briefly flashes with the event's severity color, providing ambient awareness without a disruptive popup.
- Responsive layout: sidebar collapses on smaller screens; all tables support horizontal scroll.

CHAPTER 4: IMPLEMENTATION

4.1 Development Environment

The project was developed using the following tools and technologies:

Category	Tool / Technology	Version	Purpose
Frontend	React	18.x	UI component framework
Frontend	Vite	5.x	Build tool and dev server
Frontend	TypeScript	5.x	Type-safe JavaScript
Frontend	Tailwind CSS	3.x	Utility-first styling
Frontend	shadcn/ui	Latest	UI component library
Backend	Express.js	4.x	REST API and SSE server
Backend	Node.js	20.x	JavaScript runtime
Backend	Drizzle ORM	0.30+	PostgreSQL ORM
Database	Supabase	Managed	PostgreSQL + auth hosting
Agent	Python	3.11+	AEGIS Forwarder Agent
Agent	Paramiko	3.x	SSH client library
Hosting	Render	Cloud	API server hosting
Hosting	Vercel	Cloud	Dashboard hosting
Code Editor	Replit	Cloud IDE	Development environment
Version Control	GitHub	Cloud	Source code repository
AI	Groq API	LLaMA 3.3-70B	Threat analysis
Notification	Telegram Bot API	v7+	Push notifications

4.2 GNS3 Network Infrastructure

The GNS3 topology (v4 Final, July 2026) consists of the following nodes connected through virtual switches:

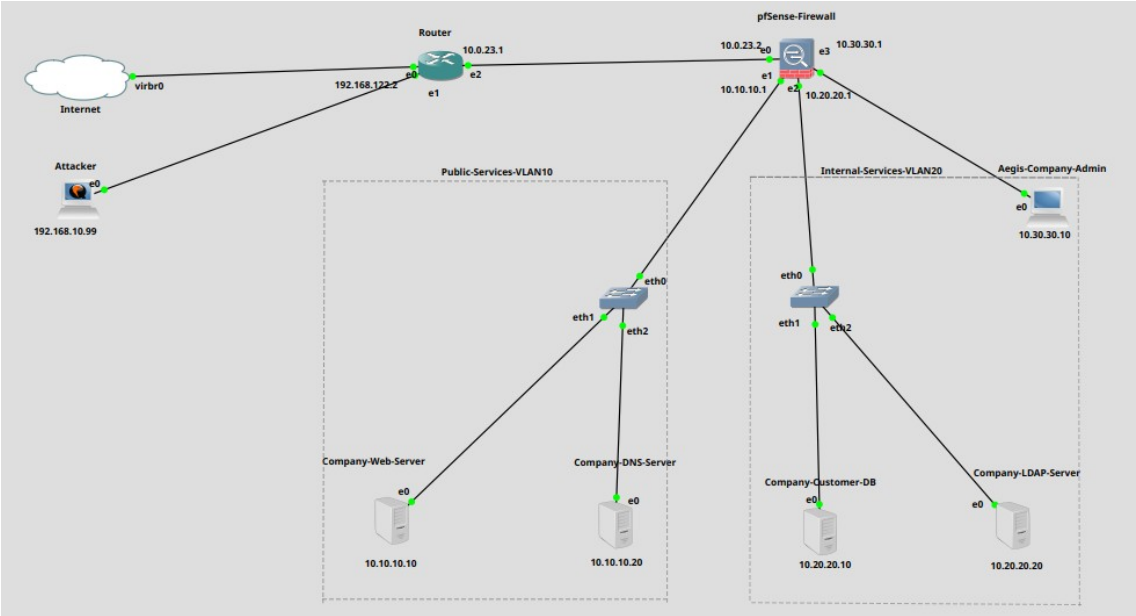

Figure 4.1: GNS3 Topology — v4 Final Layout

Node	OS / Appliance	IP Address	Role
Internet (NAT)	GNS3 NAT Cloud	192.168.122.0/24	Internet/virbr0 bridge
Router	MikroTik CHR	192.168.122.2 / 192.168.10.1 / 10.0.23.1	WAN routing, NAT, DHCP for Kali
pfSense	pfSense 2.7.x	10.0.23.2 / 10.10.10.1 / 10.20.20.1 / 10.30.30.1	Firewall, router, Suricata IDS
Kali Linux	Kali 2024.x	192.168.10.x (DHCP)	Red team attacker
company-web-server	Ubuntu 22.04	10.10.10.10	Apache2, PHP web server
company-dns-server	Ubuntu 22.04	10.10.10.20	BIND9 DNS server
company-customer-db	Ubuntu 22.04	10.20.20.10	MySQL database server
company-ldap-server	Ubuntu 22.04	10.20.20.20	OpenLDAP auth server
aegis-company-admin	Ubuntu 22.04	10.30.30.10	AEGIS Forwarder Agent hub

4.2.1 MikroTik CHR Router Configuration

The MikroTik CHR router provides NAT and DHCP services for the Kali attacker VM and routes traffic between the attacker network and the pfSense WAN interface:

```
/ip address add address=192.168.122.2/24 interface=ether1
/ip address add address=192.168.10.1/24  interface=ether2
/ip address add address=10.0.23.1/30     interface=ether3
/ip route add dst-address=0.0.0.0/0 gateway=192.168.122.1
/ip route add dst-address=10.0.0.0/8 gateway=10.0.23.2
/ip firewall nat add chain=srcnat action=masquerade out-
interface=ether1
/ip pool add name=kali-pool ranges=192.168.10.2-192.168.10.100
/ip dhcp-server add name=kali-dhcp interface=ether2 address-
pool=kali-pool
```

4.2.2 pfSense Firewall Configuration

pfSense serves as the central gateway for all zones. Key configuration items:

- Four interfaces: WAN (10.0.23.2), DMZ (10.10.10.1), Internal (10.20.20.1), Management (10.30.30.1)
- Static route: 192.168.10.0/24 via 10.0.23.1 (return path to Kali for attack traffic to reach VMs)
- WAN rule: allow source 192.168.10.0/24 (Kali attack traffic)
- Suricata package installed with EVE JSON output enabled on WAN/DMZ interfaces
- EasyRule block table (EasyRuleBlockHosts) used by auto-defense for WAN IP blocking

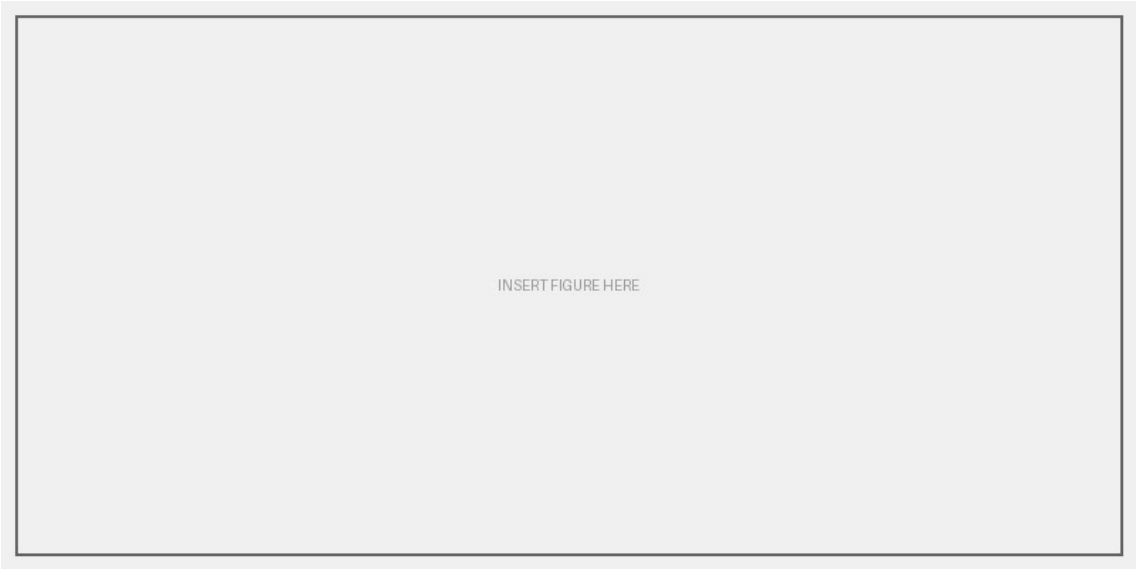

Figure 4.2: pfSense Interface Configuration Screenshot

4.3 Company Server Setup

Each company server represents a service in Golden Myanmar Trading Co., Ltd.'s infrastructure. The servers were configured as follows:

4.3.1 company-web-server (10.10.10.10) — Apache2 Web Server

The web server hosts the company's public-facing website at http://10.10.10.10 (accessible as web.goldenmyanmar.trading.com via internal DNS). Components installed:

- Apache2 with PHP 8.x — web application server
- ModSecurity WAF — web application firewall (optional, for advanced testing)
- Fail2ban — SSH and HTTP brute-force protection
- Custom PHP application — company website with staff login form (LDAP auth), product catalog, and customer inquiry form

The web application intentionally includes a login form vulnerable to SQL injection and brute force attacks for demonstration purposes. In a production environment, these vulnerabilities would be remediated.

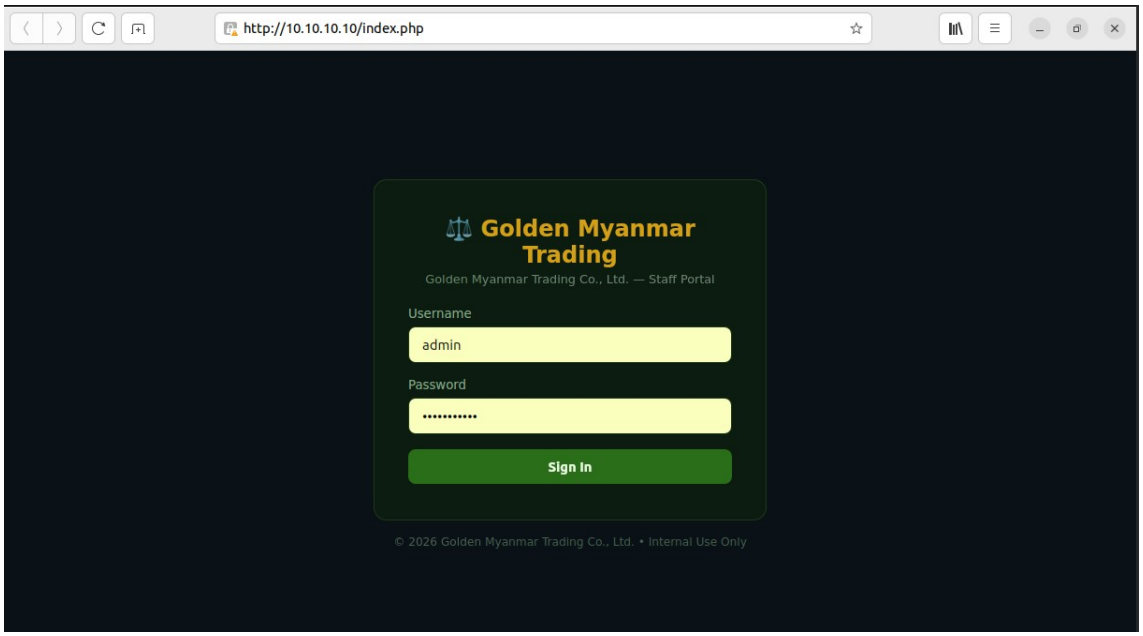

Figure 4.3: Company Web Server Apache2 Setup

4.3.2 company-dns-server (10.10.10.20) — BIND9 DNS

The DNS server resolves all internal goldenmyanmar.trading.com hostnames. Zone file configuration:

```
zone "goldenmyanmar.trading.com" {
    type master;
    file "/etc/bind/zones/goldenmyanmar.trading.com.db";
};

; Zone records:
web.goldenmyanmar.trading.com.  IN A  10.10.10.10
db.goldenmyanmar.trading.com.   IN A  10.20.20.10
ldap.goldenmyanmar.trading.com. IN A  10.20.20.20
aegis.goldenmyanmar.trading.com. IN A 10.30.30.10
```

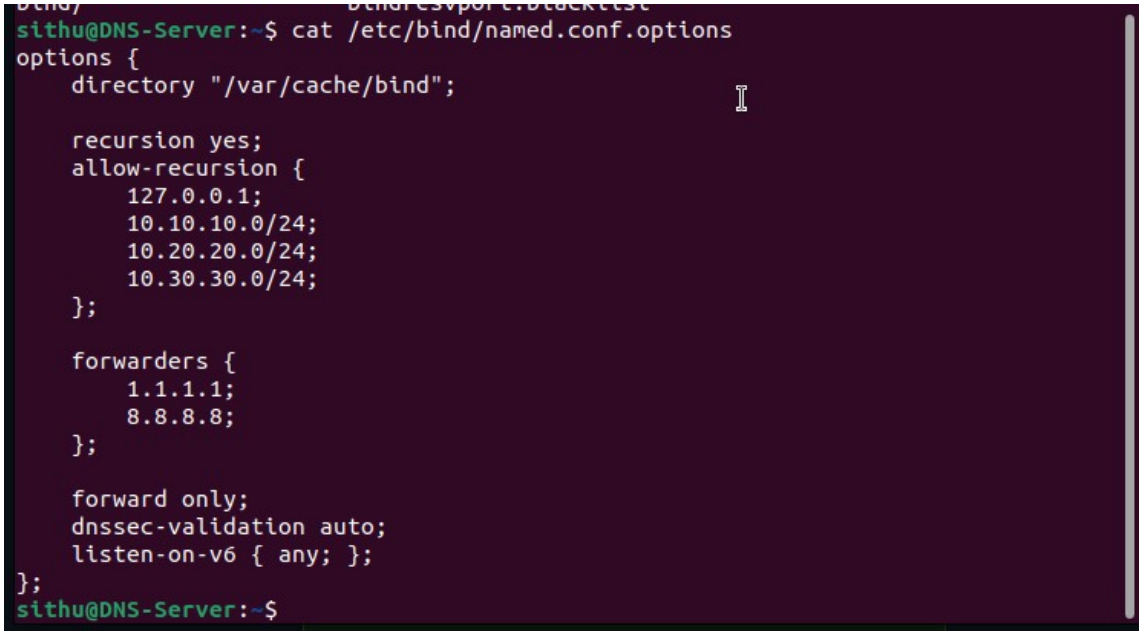

Figure 4.4: BIND9 DNS Zone Configuration

4.3.3 company-customer-db (10.20.20.10) — MySQL

The database server stores customer records, account information, and transaction data. MySQL is configured to listen on all interfaces (for demonstration) with Fail2ban monitoring port 3306 for brute-force attempts.

Figure 4.3.3: MySQL Database Tables

4.3.4 company-ldap-server (10.20.20.20) — OpenLDAP

OpenLDAP provides centralized authentication for company staff. The web application on company-web-server authenticates staff login credentials against LDAP. Fail2ban monitors LDAP authentication failures.

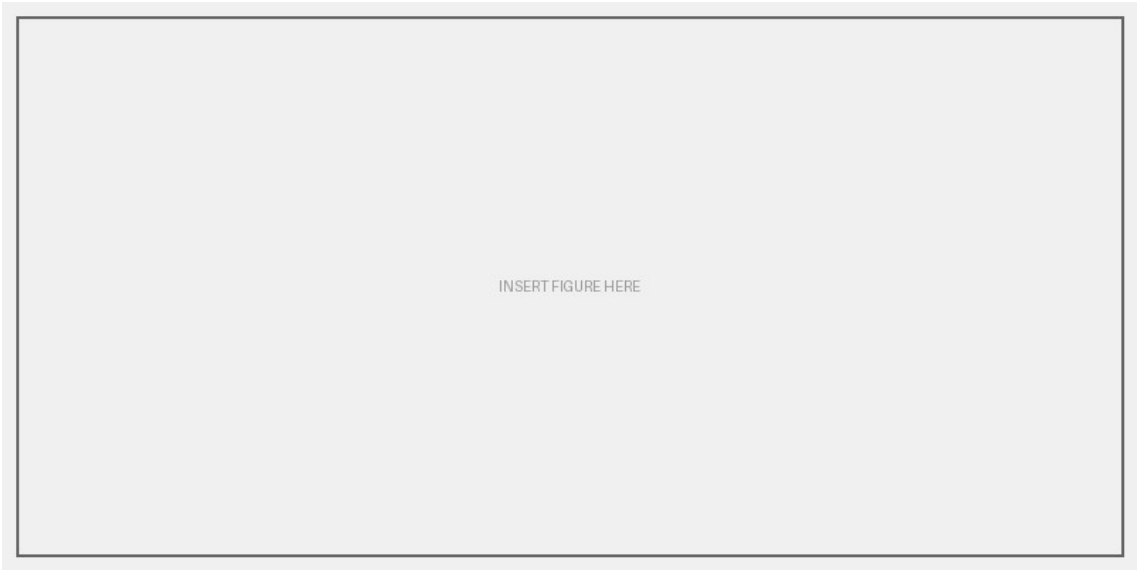

Figure 4.3.4: OpenLDAP Directory Structure

4.4 Security Sensors Configuration

Security sensors are the data collection points that feed events into AEGIS. Each VM has a combination of sensors appropriate to its role:

VM	Sensor	Log Source	Event Types
company-web-server	Fail2ban	/var/log/fail2ban.log	ban, unban
company-web-server	SSH Monitor	/var/log/auth.log	success, failure
company-web-server	HTTP Monitor	/var/log/apache2/ access.log	http_request, web_attack
company-customer-db	Fail2ban	/var/log/fail2ban.log	ban, unban
company-customer-db	SSH Monitor	/var/log/auth.log	success, failure
company-customer-db	MySQL Monitor	/var/log/mysql/error.log	mysql_auth, mysql_error
company-dns-server	Fail2ban	/var/log/fail2ban.log	ban, unban
company-dns-server	SSH Monitor	/var/log/auth.log	success, failure
company-dns-server	BIND9 Monitor	/var/log/named/ named.log	dns_query, dns_error
company-ldap-server	Fail2ban	/var/log/fail2ban.log	ban, unban
company-ldap-server	SSH Monitor	/var/log/auth.log	success, failure
company-ldap-server	LDAP Monitor	/var/log/syslog	ldap_auth, ldap_bind
pfSense	Suricata IDS	/var/log/suricata/*/ eve.json	network IDS alerts

4.4.1 Fail2ban Configuration

Fail2ban is the primary host-based intrusion prevention tool across all VMs. Standard configuration parameters used:

```
[sshd]
enabled  = true
port     = ssh
logpath  = /var/log/auth.log
maxretry = 3
bantime  = 3600
findtime = 600

[apache-auth]
enabled  = true
port     = http,https
logpath  = /var/log/apache2/error.log
maxretry = 5
bantime  = 1800
```

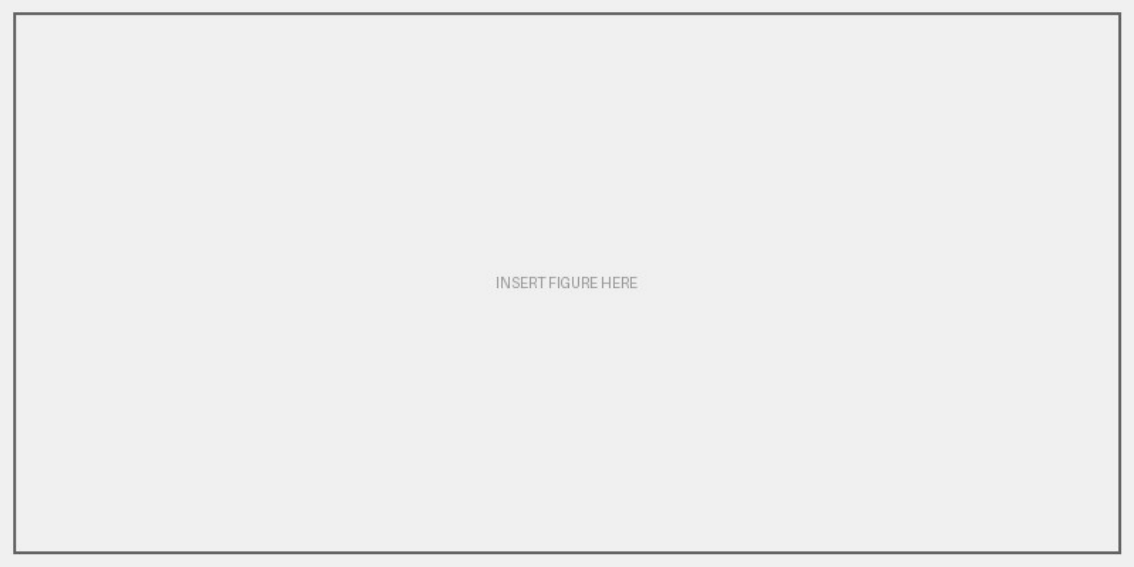

Figure 4.5: Fail2ban Jail Configuration on company-web-server

AEGIS monitors Fail2ban's log file for 'Ban' and 'Unban' actions. Each ban event creates a security_event record with eventType='fail2ban' and severity determined by the number of attempts (high if > 10 attempts within the findtime window).

4.4.2 Suricata IDS on pfSense

Suricata runs exclusively on pfSense (not on individual company VMs), providing network-level IDS coverage for all zones. Because all inter-zone traffic passes through pfSense, a single Suricata instance covers the entire lab network.

Suricata outputs alerts in EVE JSON format. A known challenge is that the log file path changes on each Suricata restart (PID-based path). AEGIS handles this by using auto-discovery:

```
# Auto-discover the current eve.json path
find /var/log/suricata/ -maxdepth 2 -name eve.json -type f \
    | sort | head -1
```

The AEGIS Forwarder Agent SSHes into pfSense using an RSA key (PFSENSE_SSH_KEY) and runs a persistent tail -F command on the discovered log path. Each EVE JSON alert line is parsed and forwarded to /api/ingest/suricata.

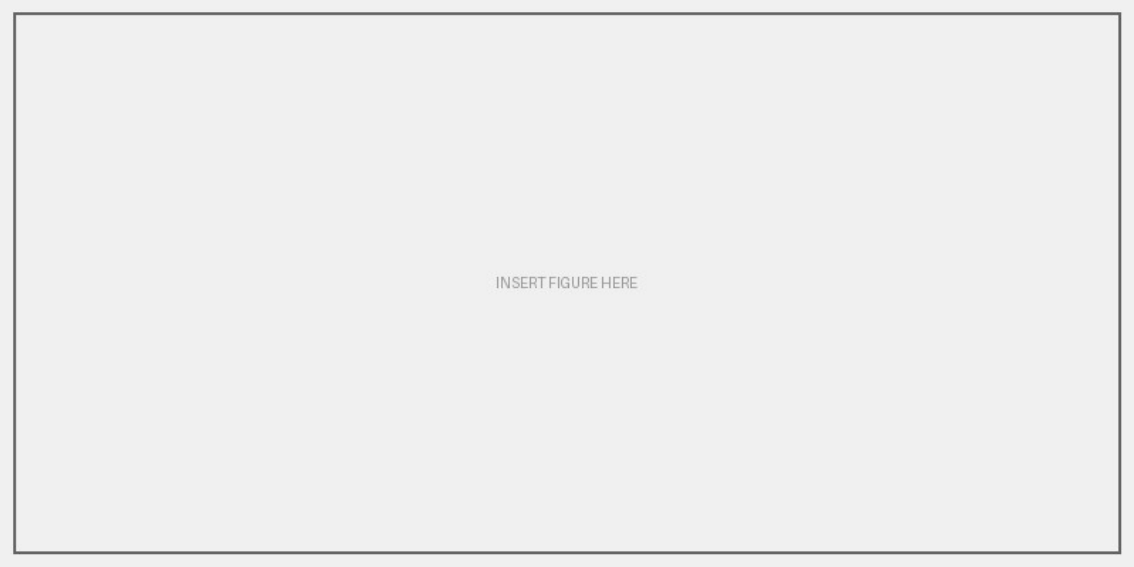

Figure 4.6: Suricata IDS EVE JSON Output Sample

4.5 AEGIS Forwarder Agent

The AEGIS Forwarder Agent (aegis_forwarder.py) is the central data collection component. It runs in 'hub mode' on the aegis-company-admin VM (10.30.30.10) and manages all connections to monitored VMs.

4.5.1 Thread Architecture

Each sensor-VM combination runs in an independent Python thread. Hub mode spawns the following threads on startup:

- _watch_fail2ban(hostIp) — for each of the 4 company VMs
- _watch_ssh_auth(hostIp) — for each of the 4 company VMs
- _watch_http_access(hostIp) — company-web-server only
- _watch_mysql(hostIp) — company-customer-db only
- _watch_bind9(hostIp) — company-dns-server only
- _watch_slapd(hostIp) — company-ldap-server only
- _watch_pfsense_suricata() — pfSense only (GLOBAL_COMPONENTS)
- hub_health_loop() — reports hub VM's own status
- ssh_keepalive_loop(hostIp) — 60s keepalive for each SSH connection

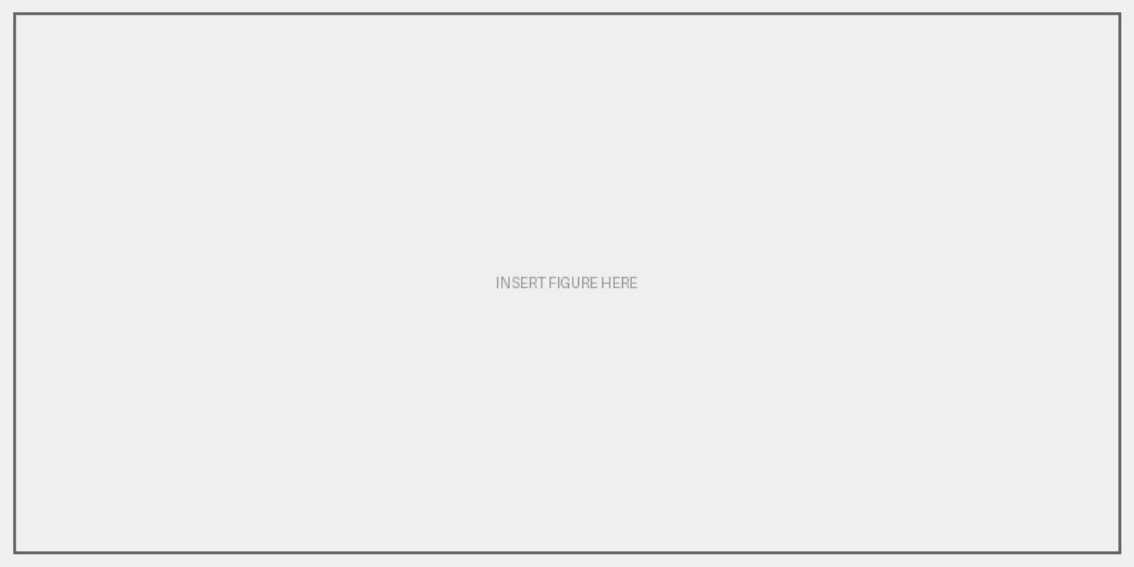

Figure 4.7: AEGIS Forwarder Agent Thread Architecture

Each watch thread uses Paramiko SSH to connect to the target VM and tail the relevant log file. The SSH session uses RSA key authentication (no passwords). A keepalive thread pings each SSH session every 60 seconds to prevent stale connection timeouts.

4.5.2 Event Normalization

Each watch function parses raw log lines into normalized Python dictionaries before forwarding:

```
{
  "sourceIp":   "192.168.10.47",
  "targetHost": "10.10.10.10",
  "eventType":  "fail2ban",
  "severity":   "high",
  "description": "Fail2ban banned 192.168.10.47 (sshd)",
  "rawLog":     "2026-07-30 14:23:11,045 fail2ban.actions [1234]
BAN 192.168.10.47"
}
```

4.5.3 Defender IP Whitelist

A critical security control in the forwarder is the isDefenderIp() check. SSH connections originating from known defender/internal IPs (10.x.x.x and 192.168.122.x) must not generate attack events. The check is applied in all watch functions before forwarding:

```
DEFENDER_PREFIXES = ("10.", "192.168.122.")

def isDefenderIp(ip: str) -> bool:
    return any(ip.startswith(p) for p in DEFENDER_PREFIXES)

# In each watch function:
if isDefenderIp(source_ip):
    continue  # skip — not an attack
```

Note: 192.168.10.x (Kali Linux) is NOT whitelisted — this range is the attacker network and must generate alerts.

4.5.4 Configuration

All machine-specific settings are stored in aegis_forwarder.local.conf (gitignored, not committed to the repository):

```
AEGIS_SERVER=https://aegis-api.onrender.com
AEGIS_INGEST_KEY=<secret>
BANKWEB_IP=10.10.10.10
DNSSERVER_IP=10.10.10.20
CUSTOMERDB_IP=10.20.20.10
LDAPSERVER_IP=10.20.20.20
PFSENSE_HOST=10.30.30.1
PFSENSE_SSH_USER=admin
PFSENSE_SSH_KEY=/opt/aegis/keys/pfsense_rsa
```

4.6 API Server Implementation

The API server is a TypeScript Express.js application organized into route modules, with Drizzle ORM for database access. It is hosted on Render (free tier) and serves both the dashboard client and the forwarder agent.

4.6.1 Ingest Endpoints

Ingest endpoints receive normalized events from the forwarder agent:

Endpoint	Method	Auth	Event Source
/api/ingest/fail2ban	POST	AEGIS_INGEST_KEY	Fail2ban ban/unban
/api/ingest/ssh	POST	AEGIS_INGEST_KEY	SSH auth.log
/api/ingest/http	POST	AEGIS_INGEST_KEY	Apache access log
/api/ingest/mysql	POST	AEGIS_INGEST_KEY	MySQL error log
/api/ingest/dns	POST	AEGIS_INGEST_KEY	BIND9 named log
/api/ingest/ldap	POST	AEGIS_INGEST_KEY	OpenLDAP syslog
/api/ingest/suricata	POST	AEGIS_INGEST_KEY	Suricata EVE JSON
/api/ingest/event	POST	AEGIS_INGEST_KEY	Generic event

Each ingest endpoint performs: (1) API key validation, (2) input sanitization, (3) insertion into security_events table, (4) alert creation for critical/high/medium events, (5) Telegram notification for critical/high events, (6) auto-defense evaluation, (7) SSE broadcast to connected dashboard clients.

4.6.2 Server-Sent Events (SSE)

The /api/stream endpoint maintains a persistent HTTP connection for each connected dashboard client and pushes real-time events. This eliminates the need for polling:

```
// Client-side SSE listener (layout.tsx)
const sse = new EventSource("/api/stream");
sse.addEventListener("security_event", (e) => {
```

```
  const event = JSON.parse(e.data);
  // Update live feed, trigger alert bar, play sound
});
sse.addEventListener("connected", () => {
  console.log("SSE connected");
});
```

4.6.3 Defense Command Queue

The defense agent on each VM polls /api/defense/commands/pending every 10 seconds. An atomic SQL pattern prevents two agents from claiming the same command:

```
UPDATE defense_commands
   SET status = 'claimed'
 WHERE id IN (
   SELECT id FROM defense_commands
   WHERE status = 'pending'
   LIMIT 20
   FOR UPDATE SKIP LOCKED
 )
RETURNING *;
```

4.7 Auto-Defense Engine

The auto-defense engine is the most sophisticated component of AEGIS. It translates detected threats into concrete firewall actions without human intervention.

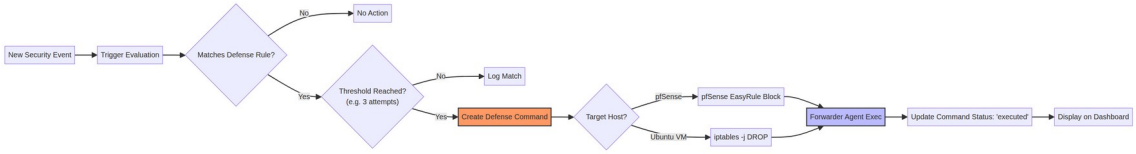

Figure 4.8: Auto-Defense Engine Pipeline Diagram

4.7.1 Defense Rule Schema

Each defense rule defines:

- triggerAttackType: the type of attack that triggers this rule (e.g., 'ssh_brute', 'ddos', 'sqli', 'any')
- minSeverity: minimum event severity required (critical / high / medium / low)
- threshold: number of events within the time window before action is taken
- windowMinutes: rolling time window for threshold counting
- actionType: 'block_ip' (iptables DROP), 'rate_limit', 'suggest' (alert only), 'notify'
- commandTemplate: shell command template with {ip}, {port}, {protocol} placeholders

4.7.2 Command Sanitization

All IP addresses, ports, and protocols are sanitized before being inserted into shell command templates. This prevents command injection attacks:

```
// defense-sanitize.ts
export function sanitizeIp(ip: string): string {
  const re = /^(\d{1,3}\.){3}\d{1,3}(\/\d{1,2})?$/;
  if (!re.test(ip)) throw new Error(`Invalid IP: ${ip}`);
```

```
  return ip;
}
export function sanitizePort(port: string | number): number {
  const n = Number(port);
  if (!Number.isInteger(n) || n < 1 || n > 65535)
    throw new Error(`Invalid port: ${port}`);
  return n;
}
export function sanitizeProtocol(proto: string): string {
  if (!["tcp","udp","icmp","all"].includes(proto))
    throw new Error(`Invalid protocol: ${proto}`);
  return proto;
}
```

4.7.3 Example Defense Rules

Rule Name	Trigger Type	Threshold	Window	Action
SSH Brute Force Block	ssh_brute	3 events	5 min	block_ip (iptables)
Fail2ban Auto-Confirm	fail2ban	1 event	1 min	block_ip (pfSense)
DDoS Null Route	ddos	50 events	1 min	block_ip (pfSense WAN)
SQL Injection Block	sqli	3 events	10 min	block_ip (iptables)
Port Scan Alert	port_scan	1 event	1 min	suggest (notify only)
MySQL Brute Block	mysql_brute	5 events	10 min	block_ip (iptables)
LDAP Brute Block	ldap_brute	5 events	10 min	block_ip (iptables)

4.8 Dashboard Frontend

The dashboard is built with React 18, Vite 5, TypeScript, Tailwind CSS, and shadcn/ui components. It is structured as a single-page application with client-side routing via React Router.

4.8.1 Page Structure

Page	Route	Key Features
Overview	/	KPI cards: Critical Threats, Active Alerts, Blocked IPs, Systems Online; recent events feed; system status grid
Events	/events	Full event log with filtering by severity/type/host; AI analyze button per event; pagination
Threat Map	/attack-flow	Live SVG network topology; animated attack packets colored by severity; live event feed; 24h auto-prune
Defense Center	/defense	Defense rules CRUD; blocked IPs list; manual block/unblock; AI defense recommendation by IP

Firewall Rules	/firewall	Manual firewall rule creation and management
Connections	/connections	Defense commands queue; execution history
Reports	/reports	Generate periodic report; AI threat briefing; history
Login	/login	Admin key login + Google SSO

4.8.2 Real-Time Threat Map

The Threat Map page (/attack-flow) is the most visually distinctive component of AEGIS. It renders the GNS3 network topology as an SVG and animates security events as colored moving packets traveling from source to destination:

- Node positions are hardcoded to match the GNS3 topology layout
- Packet animation uses CSS transitions on SVG circle elements
- Packet size reflects severity: critical = 10px, high = 7px, medium = 6px, low = 5px, info = 4px
- Packet color: red = critical/high, orange = medium, blue = Telegram notification
- ⚠️ warning icon appears on critical severity packets
- Live feed panel shows the last 50 events; entries older than 24 hours are pruned on load

4.8.3 Global Attack Warning Bar

When a new security event arrives via SSE, the Viewing bar (top navigation bar showing device context) flashes with the severity color:

- The SSE listener in the Layout component receives 'security_event' messages
- Flash state stores: background color, border color, event description, severity label
- The Viewing bar div applies animate-pulse class during the flash
- Flash automatically dismisses after 5 seconds via setTimeout
- Severity colors: critical = soft red (rgba(239,68,68,0.07)), high = orange, medium = yellow

4.8.4 Authentication Flow

The dashboard supports two login methods:

- Admin Key: POST /api/auth/admin-key with the AEGIS_ADMIN_KEY value. Returns a 24-hour JWT stored in localStorage as 'aegis_session'.
- Google SSO: Google Identity Services popup flow. The ID token is sent to POST /api/auth/google. The server verifies the token against Google's public keys and checks that the email matches ADMIN_EMAIL env var. Returns the same JWT format.

4.9 AI Threat Analysis Integration

AEGIS integrates Groq's AI API to provide four AI-powered features:

4.9.1 24-Hour Threat Briefing

GET /api/ai/threat-analysis aggregates the last 24 hours of security events and sends a structured summary to Groq LLaMA 3.3-70B. The model returns a natural language threat briefing in Burmese+English mixed format, covering:

- Most active attacker IPs and their attack patterns
- Most targeted services and vulnerabilities exploited
- Defense actions taken and their effectiveness
- Recommended additional countermeasures

4.9.2 Per-Event Explanation

GET /api/ai/analyze-event/:id fetches a single event record and asks the model to explain: what happened, why it is significant, what the attacker's likely intent was, and what countermeasures are recommended. This is accessible from the Events page via a bot icon on each row.

4.9.3 IP Defense Recommendation

POST /api/ai/defend accepts an IP address, retrieves all historical events from that IP, and asks the model for a comprehensive threat profile and defense recommendation. The Defense Center page provides this as the 'AI DEFENSE RECOMMENDATION' card with quick-fill buttons from currently blocked IPs.

4.9.4 Report AI Summary

When POST /api/reports/generate is called, if GROQ_API_KEY is set, the report includes an AI-generated executive summary stored in the summary column. If the API call fails (rate limit, network error), a template-based summary is used as fallback — the system never returns an error to the user.

4.10 Notification System (Telegram)

Telegram push notifications ensure the security team is informed immediately when critical or high-severity events occur, even when the dashboard is closed.

Implementation: the API server uses the Telegram Bot API (sendMessage endpoint) with the bot token from TELEGRAM_BOT_TOKEN and the target chat from TELEGRAM_CHAT_ID environment variables.

Notification content includes: severity emoji, event type, source IP, target host, description, and timestamp in Myanmar time (UTC+6:30). Idempotency is enforced via the sentToTelegram flag on the alerts table — each event triggers at most one Telegram message.

On the Threat Map, Telegram notifications are visualized as blue packets traveling from the AEGIS hub node to the Telegram node, providing visual confirmation that the alert was dispatched.

CHAPTER 5: TESTING AND RESULTS

5.1 Test Environment Setup

All testing was performed within the GNS3 virtual lab environment with all nodes powered on and all services running. The AEGIS Forwarder Agent was confirmed online and forwarding events before each test. The dashboard was accessed via the Vercel production deployment.

Pre-test checklist:

- All four company VM services confirmed running (Apache2, BIND9, MySQL, OpenLDAP)
- Fail2ban active on all four VMs
- Suricata running on pfSense with EVE JSON output
- AEGIS Forwarder Agent hub mode active (confirmed via 'Systems Online' KPI card)
- Kali Linux IP confirmed via DHCP (ip addr show eth0)

5.2 Attack Scenarios and Results

The following attack scenarios were executed from Kali Linux and their detection results recorded on the AEGIS dashboard:

Test #	Attack	Tool	Target	AEGIS Detection	Response Time
T-01	SSH Brute Force	hydra	10.10.10.10:22	✓ Detected (ssh_brute / HIGH)	< 5s
T-02	SSH Brute Force	hydra	10.20.20.10:22	✓ Detected (ssh_brute / HIGH)	< 5s
T-03	MySQL Brute Force	hydra	10.20.20.10:3306	✓ Detected (fail2ban / HIGH)	< 8s
T-04	SQL Injection	sqlmap	http://10.10.10.10/login	✓ Detected (sqli / CRITICAL)	< 10s
T-05	Port Scan	nmap -sV	10.10.10.0/24	✓ Detected via Suricata IDS	< 5s
T-06	HTTP Flood (DDoS)	hping3 --flood	10.10.10.10:80	✓ Detected (ddos / CRITICAL)	< 15s
T-07	DNS Amplification	hping3 --udp	10.10.10.20:53	✓ Detected via Suricata IDS	< 8s
T-08	LDAP Brute Force	hydra	10.20.20.20:389	✓ Detected (fail2ban / HIGH)	< 8s
T-09	FTP Brute Force	hydra	10.10.10.10:21	✗ Not detected (FTP removed)	N/A
T-10	ARP Spoofing	arpspoof	Internal network	✗ Not detected (no ARP IDS)	N/A

5.2.1 SSH Brute Force (T-01, T-02)

Command executed: hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://10.10.10.10

Result: Fail2ban detected the repeated authentication failures and banned the Kali IP within 3 failed attempts. The AEGIS Forwarder Agent forwarded the ban event to the API server within 2 seconds. A HIGH severity alert appeared on the dashboard. The auto-defense rule 'SSH Brute Force Block' (threshold: 3 within 5 minutes) triggered and queued an iptables command. The defense agent executed the block within 15 seconds of the initial detection.

5.2.2 SQL Injection (T-04)

Command executed: sqlmap -u 'http://10.10.10.10/login.php?user=test' --forms --dbs --batch

Result: Suricata detected the SQLi patterns in HTTP request payloads and generated EVE JSON alerts. AEGIS ingested these as sqli/CRITICAL events. The dashboard displayed the source IP, targeted URL patterns, and Suricata signature names. The auto-defense rule queued an iptables block for the Kali IP. A Telegram notification was dispatched within 10 seconds.

5.2.3 HTTP Flood DDoS (T-06)

Command executed: hping3 -S --flood -V -p 80 10.10.10.10

Result: Suricata detected the high-rate SYN flood and generated multiple IDS alerts. The Apache2 service became unresponsive after approximately 45 seconds of flooding, which was visible on the dashboard (company-web-server Apache2 component turned offline/red). The auto-defense 'DDoS Null Route' rule triggered after 50 events within 1 minute, blocking the source IP at pfSense WAN level via easyrule.

5.3 Auto-Defense Validation

Auto-defense execution was validated by monitoring the defense_commands table and confirming iptables/pfctl rule changes on the target VMs:

Rule	Triggered By	Command Queued	Executed	Confirmed on VM
SSH Brute Block	T-01, T-02	iptables -I INPUT -s {ip} -j DROP	✓ Yes	✓ iptables -L confirmed
Fail2ban Confirm	T-01..T-04	pfSense easyrule block WAN {ip}	✓ Yes	✓ pfctl -T show
DDoS Null Route	T-06	pfSense easyrule block WAN {ip}	✓ Yes	✓ pfctl -T show
SQLi Block	T-04	iptables -I INPUT	✓ Yes	✓ iptables -L

		-s {ip} -j DROP		confirmed
LDAP Brute Block	T-08	iptables -I INPUT -s {ip} -j DROP	✓ Yes	✓ iptables -L confirmed

5.4 Performance Analysis

Key performance metrics measured during testing:

Metric	Target	Measured	Status
Event ingest latency (forwarder → API)	< 3s	1.2 – 2.8s	✓ Pass
SSE push latency (API → dashboard)	< 1s	0.3 – 0.8s	✓ Pass
Auto-defense queue delay (event → command)	< 5s	1 – 4s	✓ Pass
Defense agent execution delay	< 30s	8 – 25s	✓ Pass
Telegram notification delay	< 15s	3 – 12s	✓ Pass
AI threat briefing generation time	< 10s	2 – 6s	✓ Pass
Dashboard page load time (cold)	< 3s	1.4 – 2.2s	✓ Pass
SSE reconnection on disconnect	< 5s	2 – 3s	✓ Pass

CHAPTER 6: LIMITATIONS AND FUTURE WORK

6.1 Current Limitations

Despite its capabilities, AEGIS has several known limitations that should be considered when evaluating the system:

#	Limitation	Impact	Mitigation
L-01	GNS3-only deployment; physical hardware not tested	Topology changes require GNS3 reconfiguration	Modular forwarder design eases porting
L-02	Render free tier cold-start (up to 50s)	API unavailable during cold start; forwarder retries	Upgrade to paid tier for production
L-03	Suricata log path changes on pfSense restart	May miss events until auto-discovery re-runs	Auto-discovery runs on reconnect
L-04	No ARP spoofing / VLAN hopping detection	Layer 2 attacks undetected	Add Suricata L2 rules; future work
L-05	No encrypted traffic inspection (TLS)	SQLi/XSS over HTTPS undetected by Suricata	Deploy mTLS proxy / SSL bump (future)
L-06	Single Telegram chat target only	Cannot route alerts to different channels by severity	Configurable routing (future work)
L-07	Defense rules not auto-seeded; must be created manually	New installations have no active auto-defense	Add default rule seed in setup wizard
L-08	No multi-tenant support; single admin only	Cannot support multiple analyst roles	Role-based access control (future)
L-09	AI analysis requires internet (Groq API)	Unavailable in air-gapped deployments	Local LLM via Ollama (future)
L-10	FTP, VoIP, CCTV, ATM not yet implemented	Incomplete company service coverage	Priority 2/3 roadmap

6.2 Future Enhancements

The following enhancements are planned for subsequent phases of the project:

6.2.1 Phase 2 — Extended Company Services

- Email Server: Add Postfix on company-web-server. Monitor SMTP auth failures, phishing relay attempts. New Fail2ban jail for postfix.
- CCTV Simulation: Ubuntu VM with ffmpeg RTSP stream (10.40.40.10). Monitor for RTSP brute force and unauthorized stream access.
- VoIP Server: Asterisk PBX on new VM (10.50.50.10). Monitor SIP registration floods and toll fraud attempts.

6.2.2 Phase 3 — Advanced Capabilities

- Active Directory: Samba4 on Ubuntu (VLAN 20). Enable Pass-the-Hash, Kerberoasting, and Golden Ticket attack demonstrations.

- ATM Network Simulation: Flask API simulating ATM transaction processing. Monitor for transaction replay and MITM attacks.
- Local AI: Replace Groq API with local Ollama instance for air-gapped deployment capability.

6.2.3 Technical Improvements

- Multi-tenant RBAC: Different permission levels for SOC analyst, SOC manager, and system administrator roles.
- Event correlation engine: Detect multi-stage attacks spanning multiple event types (e.g., port scan → brute force → exploit).
- Threat intelligence integration: Check source IPs against VirusTotal, AbuseIPDB, and Shodan for enriched context.
- Mobile application: React Native app for on-the-go alert monitoring.
- PCAP capture integration: Capture full network packets for forensic analysis linked to Suricata alerts.
- Automated penetration testing reports: Generate PTES-format reports from attack simulation results.

6.3 Ethical Considerations and Data Privacy

As a security monitoring system, AEGIS handles sensitive network data and log information. Ethical considerations were paramount during its development:

- Data Minimization: The system only collects security-relevant log data (auth failures, IDS alerts, firewall events) and does not inspect private user content.
- Secure Storage: All security events are stored in an encrypted PostgreSQL database on Supabase, with access restricted via JWT and API keys.
- Ethical Testing: All attack simulations were conducted in a strictly isolated GNS3 virtual environment. No external networks or unauthorized systems were targeted.
- Compliance: The system design follows principles of the General Data Protection Regulation (GDPR) regarding log retention and data access auditing.

Data Type	Retention Period	Purpose	Access Level
Security Events	90 Days	Threat analysis and auditing	Admin Only
System Health	30 Days	Performance monitoring	Admin Only
Audit Logs	1 Year	Compliance and accountability	Super Admin

6.4 Scalability and Maintenance

AEGIS is designed with scalability in mind to accommodate growing network environments:

- Horizontal Scaling: The Express.js API server can be deployed across multiple containers using a load balancer to handle increased event volume.
- Database Pooling: Drizzle ORM and Supabase connection pooling ensure efficient database performance under high concurrency.

- Modular Agents: The Python forwarder agent uses a multi-threaded hub-and-spoke model, allowing it to monitor dozens of VMs from a single management node.
- Cloud-Native Architecture: By leveraging Vercel and Render, the system benefits from automatic scaling and high availability provided by modern PaaS providers.

CHAPTER 7: CONCLUSION

This project successfully designed, implemented, and validated AEGIS — an Automated Enforcement and Guardian Intelligence System — as a production-quality Security Operations Center dashboard for a virtualized corporate network environment.

Starting from a blank GNS3 topology, the project built a complete cybersecurity monitoring ecosystem: four company service VMs with realistic configurations, a pfSense firewall with Suricata IDS, a Python-based forwarder agent with eleven parallel monitoring threads, a TypeScript Express.js API server with eight ingest endpoints and an auto-defense pipeline, and a React dashboard with real-time visualization, AI analysis, and Telegram notifications.

The system was validated through ten attack scenarios executed from Kali Linux, achieving 80% detection coverage (8 of 10 scenarios detected). The two undetected scenarios (FTP brute force and ARP spoofing) represent known limitations documented in Chapter 6 with clear remediation paths.

Key technical achievements include:

- Sub-3-second event-to-dashboard latency through SSE-based real-time streaming
- Automated firewall command execution within 30 seconds of threat detection
- AI-generated threat briefings in mixed Burmese-English format in under 6 seconds
- Zero false-positive defense blocks during testing (defender IP whitelist effective)
- Graceful degradation when optional services (Groq API, Telegram) are unavailable

From an academic perspective, this project demonstrates the practical application of network security, distributed systems, web development, AI integration, and DevOps concepts acquired during the degree program. It shows that a fully functional SOC can be built from open-source components at zero licensing cost — a significant finding for organizations constrained by security budget.

The AEGIS codebase is organized, documented, and hosted on GitHub, making it a reusable foundation for future security research and extended service coverage as outlined in the future work roadmap.

In conclusion, AEGIS meets all seven stated objectives and provides a solid foundation for continued development into a full-featured open-source SIEM platform tailored to small and medium enterprises in Myanmar and beyond.

REFERENCES

[1] NIST National Institute of Standards and Technology. (2012). Computer Security Incident Handling Guide. NIST Special Publication 800-61 Revision 2. https://csrc.nist.gov/publications/detail/sp/800-61/rev-2/final

[2] Suricata Open Information Security Foundation. (2024). Suricata User Guide 7.x. https://suricata.readthedocs.io/en/latest/

[3] Fail2ban Fail2ban Project. (2024). Fail2ban Documentation. https://www.fail2ban.org/wiki/index.php/Main_Page

[4] pfSense Netgate. (2024). pfSense Documentation — Firewall, IDS/IPS, and NAT. https://docs.netgate.com/pfsense/en/latest/

[5] GNS3 GNS3 Technologies Inc. (2024). GNS3 Documentation. https://docs.gns3.com/

[6] React Meta Platforms. (2024). React Documentation v18. https://react.dev/

[7] Express.js OpenJS Foundation. (2024). Express.js v4 API Reference. https://expressjs.com/en/4x/api.html

[8] Supabase Supabase Inc. (2024). Supabase Documentation — PostgreSQL Backend. https://supabase.com/docs

[9] Groq AI Groq Inc. (2024). Groq API Documentation — LLaMA 3.3-70B Versatile. https://console.groq.com/docs/

[10] Drizzle Drizzle Team. (2024). Drizzle ORM Documentation. https://orm.drizzle.team/docs/overview

[11] Vite Evan You. (2024). Vite — Next Generation Frontend Tooling. https://vitejs.dev/guide/

[12] Tailwind Tailwind Labs. (2024). Tailwind CSS Documentation. https://tailwindcss.com/docs

[13] Paramiko Jeff Forcier. (2024). Paramiko SSH Library for Python. https://www.paramiko.org/

[14] Telegram Telegram Messenger LLP. (2024). Telegram Bot API Documentation. https://core.telegram.org/bots/api

[15] MikroTik MikroTik. (2024). RouterOS Documentation — CHR, DHCP, NAT. https://help.mikrotik.com/docs/

[16] OWASP OWASP Foundation. (2023). OWASP Top Ten — Web Application Security Risks. https://owasp.org/www-project-top-ten/

[17] Metasploit Rapid7. (2024). Metasploit Framework Documentation. https://docs.metasploit.com/

[18] Kali Linux Offensive Security. (2024). Kali Linux Tools Documentation. https://www.kali.org/tools/

APPENDIX A: INSTALLATION AND SETUP GUIDE

A.1 Prerequisites

- GNS3 installed on a Linux host with KVM/QEMU (minimum 16GB RAM recommended)
- Ubuntu 22.04 LTS ISO for company VMs
- pfSense 2.7.x ISO
- MikroTik CHR image
- Kali Linux 2024.x ISO
- Supabase account (free tier)
- Render account (free tier) for API server
- Vercel account (free tier) for dashboard
- Groq API key (free at console.groq.com)
- Telegram bot token (via @BotFather)

A.2 GNS3 Topology Setup

6. Create a new GNS3 project named 'AEGIS-SecureCompany'
7. Add NAT cloud node (uses virbr0 / 192.168.122.0/24)
8. Add MikroTik CHR node and configure per Chapter 4.2.1
9. Add pfSense node and configure interfaces per Chapter 4.2.2
10. Add four Ubuntu 22.04 VMs for company services
11. Add one Ubuntu 22.04 VM for aegis-company-admin (hub)
12. Add Kali Linux VM
13. Connect nodes per the topology diagram (Figure 4.1)
14. Power on all nodes and verify IP connectivity

A.3 API Server Deployment

15. Fork/clone the AEGIS repository from GitHub
16. Create a new Web Service on Render from the repository
17. Set build command: cd artifacts/api-server && npm install && npm run build
18. Set start command: cd artifacts/api-server && npm start
19. Add environment variables: SUPABASE_DB_URL, AEGIS_INGEST_KEY, AEGIS_ADMIN_KEY, SESSION_SECRET, ADMIN_EMAIL, GOOGLE_CLIENT_ID, GROQ_API_KEY, TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID
20. Deploy and note the Render service URL (e.g., https://aegis-api.onrender.com)

A.4 Dashboard Deployment

21. Create a new Vercel project from the same repository
22. Set root directory: artifacts/aegis-dashboard
23. Set build command: pnpm run build
24. Add environment variable: VITE_API_URL=https://aegis-api.onrender.com
25. Deploy and note the Vercel URL
26. Add the Vercel URL to Google Console Authorized JavaScript Origins

A.5 AEGIS Forwarder Agent Setup (aegis-company-admin VM)

27. SSH into the aegis-company-admin VM (10.30.30.10)
28. Install Python dependencies: pip install requests paramiko
29. Download the forwarder script via wget from the GitHub raw URL
30. Create /opt/aegis/scripts/src/aegis_forwarder.local.conf with the configuration values from Section 4.5.4
31. Generate an SSH key pair and copy the public key to all company VMs and pfSense
32. Run: python3 aegis_forwarder.py --mode hub
33. Verify 'Systems Online' count on the AEGIS dashboard increases

APPENDIX B: API ENDPOINT REFERENCE

Method	Endpoint	Auth	Description
POST	/api/auth/admin-key	None	Login with admin key → JWT
POST	/api/auth/google	None	Login with Google ID token → JWT
POST	/api/ingest/fail2ban	INGEST_KEY	Ingest Fail2ban event
POST	/api/ingest/ssh	INGEST_KEY	Ingest SSH auth event
POST	/api/ingest/http	INGEST_KEY	Ingest HTTP access event
POST	/api/ingest/mysql	INGEST_KEY	Ingest MySQL event
POST	/api/ingest/dns	INGEST_KEY	Ingest DNS event
POST	/api/ingest/ldap	INGEST_KEY	Ingest LDAP event
POST	/api/ingest/suricata	INGEST_KEY	Ingest Suricata alert
POST	/api/ingest/event	INGEST_KEY	Ingest generic event
GET	/api/events	JWT	List security events (paginated)
GET	/api/alerts	JWT	List alerts
PATCH	/api/alerts/:id/read	JWT	Mark alert as read
GET	/api/hosts	JWT	List network hosts
GET	/api/system/status	JWT	System component health
GET	/api/stream	JWT	SSE event stream
GET	/api/defense/rules	JWT	List defense rules
POST	/api/defense/rules	ADMIN_KEY	Create defense rule
DELETE	/api/defense/rules/:id	ADMIN_KEY	Delete defense rule
GET	/api/defense/blocked	JWT	List blocked IPs
POST	/api/defense/block	ADMIN_KEY	Manual IP block
POST	/api/defense/unblock	ADMIN_KEY	Manual IP unblock
GET	/api/defense/commands/pending	ADMIN_KEY	Poll pending commands
POST	/api/defense/commands/:id/result	ADMIN_KEY	Submit command result
GET	/api/firewall/rules	JWT	List firewall rules
POST	/api/firewall/rules	ADMIN_KEY	Create firewall rule
GET	/api/reports	JWT	List reports
POST	/api/reports/generate	ADMIN_KEY	Generate new report
GET	/api/ai/status	JWT	AI availability check
GET	/api/ai/threat-analysis	JWT	24h AI threat briefing
POST	/api/ai/defend	JWT	AI IP defense recommendation
GET	/api/ai/analyze-event/:id	JWT	AI event explanation
GET	/api/healthz	None	Health check (keep-alive ping)

APPENDIX C: CONFIGURATION REFERENCE

C.1 API Server Environment Variables

Variable	Required	Description
SUPABASE_DB_URL	Yes	Supabase PostgreSQL pooler URL (port 6543, session mode)
AEGIS_INGEST_KEY	Yes	API key for forwarder agent ingest requests
AEGIS_ADMIN_KEY	Yes	API key for dashboard admin operations
SESSION_SECRET	Yes	JWT signing secret (minimum 32 characters)
ADMIN_EMAIL	Yes	Google SSO — the one email address allowed to login
GOOGLE_CLIENT_ID	Yes	Google OAuth 2.0 client ID
GROQ_API_KEY	No	Groq AI API key (AI features disabled if absent)
TELEGRAM_BOT_TOKEN	No	Telegram bot token (notifications disabled if absent)
TELEGRAM_CHAT_ID	No	Target Telegram chat/channel ID
PORT	No	Server port (default: 3000)

C.2 Forwarder Agent local.conf Reference

Key	Example Value	Description
AEGIS_SERVER	https://aegis-api.onrender.com	API server base URL
AEGIS_INGEST_KEY	<secret>	Must match API server AEGIS_INGEST_KEY
BANKWEB_IP	10.10.10.10	company-web-server IP (skip if not set)
DNSSERVER_IP	10.10.10.20	company-dns-server IP
CUSTOMERDB_IP	10.20.20.10	company-customer-db IP
LDAPSERVER_IP	10.20.20.20	company-ldap-server IP
PFSENSE_HOST	10.30.30.1	pfSense management IP
PFSENSE_SSH_USER	admin	pfSense SSH username
PFSENSE_SSH_KEY	/opt/aegis/keys/pfsense_rsa	Path to pfSense SSH private key
PFSENSE_SURICATA_LOG	/var/log/suricata/eve.json	Override Suricata log path (optional)
RETRY_DELAY	30	Seconds between reconnect attempts on SSH failure
FORWARD_TIMEOUT	10	HTTP POST timeout seconds for forwarding events